

AI in Healthcare Innovation: What to Act On, What to Watch, What to Plan Against

Advice for MedTech and Digital Medicine founders facing 5 to 10 years from concept to commercial deployment while AI capability changes on a much shorter clock.

This is my own opinion. It does not represent the views of any institution I am affiliated with. It is a suggested planning framework, not regulatory, legal, or investment advice.

Version 1.0. Last reviewed September 2026.

Comments are welcome, anonymously if you prefer. The questions I am asking are at the end.
Contact: jcollins11@mgh.harvard.edu

What I see

I work with a lot of medical device and digital medicine start-up teams. Nearly every one of them has raised some version of the same worry. AI is moving faster than their program can, the product will take years to reach the market, and by the time it arrives something will do the job better.

The anxiety is close to uniform across teams, but the actual exposure is not. Some of the teams with the least to fear worry the most, and some of the teams that should be worried are not. Separately, founders consistently underestimate how long it will take to reach the market. Timelines vary widely, and a device on a well-worn predicate can move fast. But across the early-stage programs I work with, five to ten years from concept to commercial deployment is common, and the last stretch, from clearance to someone actually paying and using the product, is the part most consistently left out of the plan.

Those two facts compound. If you misjudge your exposure and underestimate your timeline, you optimize against the wrong risk for longer than you planned to.

The worry teams bring me is usually about the product. Will something make our device obsolete? Sometimes, yes. Obsolescence risk concentrates where a product's value rests on a task that is becoming easy, or on one that may stop being necessary. Whether yours does is not settled by how worried you feel about it, and there is a test for it further down.

That is the narrower half. The larger half is that the environment the product launches into will not be the one it was designed for. The clinicians using it, the evidence expected of it, and the workflow it sits in are all moving, and none of that requires your product to contain any AI. Only part of one of the three layers below is about the technology itself. Everything else applies whatever your product type.

I have explored this issue for the past year. What follows is the framework I am now using with teams. Most of the individual ideas here are borrowed and adapted. Appendix C says from whom. What is missing elsewhere is a version a medical device or digital medicine team can run against its own program, from concept through market, which is what I have tried to build. I am publishing it to have it critiqued.

Also, I have seen confusion created by inconsistent language, and one term causes more of it than the rest. "The model" gets used for what general-purpose systems can do at all, for a supplier's product, for the piece you ship, and for the regulated device. Those are different things with different owners, and collapsing them into one word hides which of them your business actually depends on. Appendix A sets out the vocabulary I use here. If you use the Healthcare Business Model Canvas ([H-BMC](#)) framework, Appendix B maps every section onto the blocks it tests.

The instinct to forecast

When the environment moves, the instinct is to predict where it lands. Resist it.

Nobody in this field has a demonstrated record of accurate three year prediction about AI capability. A plan anchored to a specific prediction fails in two distinct ways. It fails when the prediction is wrong. It also fails when the prediction is right but arrives two years off schedule, because the capital plan and the evidence plan were timed to it.

The alternative is not better forecasting. It is to stop asking how and when things will change and start asking what you are exposed to if they do. Sort your assumptions by what you can do about each one. Some you can act on today, some will shift under you whatever you do, and some will move slowly enough to watch. Each of those takes a different response, and applying one response to all three is the mistake this is meant to prevent.

These are questions you can answer today, from what you already know about your own product. Every assumption in your business is exposed to change on a different clock, and you have a different amount of control over each one. Sort them on those two properties and you get three action layers, each of which takes a different response.

Sorting your exposure: the three layers

There are three exposure layers, and each asks a different action of you. Your programme runs through all three at once, for its whole life. These are not stages.

Figure 1: The three exposure layers



Fast layer assumptions cannot carry a plan. Nobody knows what they will be, and building dates and money on them produces a forecast wearing a plan's clothes. Slow layer assumptions can carry a plan, provided you treat them as assumptions and not as facts, which is what the observables are for. And a plan that ignores the immediate layer leaves money and months on the table for no reason at all.

The immediate layer: use the tools on your own development process now

This is the only layer where the capability already exists and you control it. That is what makes waiting the error rather than the safe choice. Everywhere else in this article the honest answer is that you cannot act yet, only decouple or watch. Here you can act today, and the return can be measured in months of program time.

Your competitors have access to the same tools on the same day, so the access itself is not an advantage. What may not be equal is the ability to find a new capability, judge where it fits, and put it into regulated work without breaking anything. That is an organizational skill rather than a purchase, it takes time to build, and a competitor cannot buy it or easily tell how long it would take them. Whether it lasts, I do not know. But it is the one thing in this layer that a competitor cannot get on the same day you do.

The tools are already competent at work that sits on the critical path of a device program: predicate identification and comparison tables, draft submission sections, risk management files and hazard analysis, human factors documentation, literature search and appraisal, design history file assembly and traceability matrices, test harnesses and edge case enumeration, protocol and statistical analysis plan drafting, informed consent language, payer landscape and reimbursement code research, and continuous monitoring of competitor clearances, guidance revisions and coverage determinations. That last one is the work the tracking section below depends on, and the reason it is now affordable.

Adopting these tools is well covered elsewhere and I am not going to add to it. The point here is narrower and belongs to the sorting.

One category is missing from the list above. Everything in it accelerates work you already know how to do. The harder application is to the commercialization reasoning itself. What your interview evidence actually supports. Which business model claims are still unsubstantiated. Where the gap sits between what you believe and what you can show. That work usually happens in a founder's head or in a slide deck, and it is where the expensive errors are made. It is also the harder thing to build well, because a tool that produces confident business model text without tracing it to evidence makes the problem worse rather than better. Ask of anything in this category what its output rests on, and whether it will tell you when the answer is nothing.

One example, and its limits. This article is itself a product of the immediate layer. Working with Claude, I researched and wrote it in less than about half the time it would have taken alone. More usefully, the process surfaced ideas I would not have considered, let me check a fact without losing the thread, and caught inconsistencies and language that would have confused a reader. What changed was not the writing. It was having something that argued with the structure, held me

to one vocabulary, and helped me find where a claim needed evidence and where it existed. That last one is the same work a hazard analysis or a predicate comparison needs, which is why I think this transfers. The limit is that none of it went into a regulated file, and nothing here tells you how it behaves when it does.

Three conditions on using the tools in healthcare innovation:

1. **GPAI output is a draft and never a deliverable.** Where the work enters a regulated file, review, ownership, and sign-off do not change, and the responsible individual is still the responsible individual. Where it does not, the same rule applies with none of the machinery to enforce it, which is the more dangerous case. Verify every citation, standard, and regulatory reference against the source. A fabricated reference in a regulated file is worse than no reference, and a fabricated one in a market analysis is simply invisible until it costs you something. Note also that a GPAI Model used as an internal drafting tool sits outside the device and is not SOUP. A GPAI Model inside the product is a different question, covered below under supplier dependency. The two are easy to conflate, and conflating them costs you in both directions: internal use gets restricted where it did not need to be, and product use gets controlled less than it should.
2. **The gain is only real if the plan changes.** If you adopt the tools and keep the same headcount plan, the same milestone dates, and the same raise, you have captured nothing. Restate the plan on one of three bases: a smaller team, earlier milestones, or the same raise buying more evidence. Pick one and put it in the deck.
3. **Control the environment before you control the work.** There is a real difference between a deployment where retention, training on your inputs, and data residency are set by your contract, and a consumer account where they are set by whatever the supplier's terms say this month. Decide which one your team is using, then decide what may go into it. The categories that need a decision rather than a habit are patient data and anything carrying HIPAA obligations, confidential clinical data, unpublished invention disclosures where a public posting can affect patentability, source code and specifications held as trade secrets, and anything you hold under someone else's confidentiality terms. One line per category is enough. A policy nobody reads is the same as no policy.

The fast layer: what shifts under you

Three separate things move in the fast layer, and they are not alike. What general-purpose systems can do at all. The terms on which you can buy access to one. And how a specific version behaves after its supplier updates it. You control none of them, none is predictable more than a few quarters out, and every competitor receives the same change on the same day.

1. **Capability envelope.** What general-purpose AI systems can do at all. Tasks that today require a competent practitioner to make a decision, to produce and communicate information, or to carry out a skilled physical task, become commoditized without warning. From a product team's point of view the envelope can expand abruptly: a single release can make a previously hard task routine, which means the expansion cannot be scheduled and cannot be extrapolated from last year's rate. Robotic capability sits across two layers rather than in one. The control policy is

software and moves on this clock. The physical actuator, instrument, sterilization method, capital cost, and the clinician training do not, and they are listed in the slow layer below.

2. **Supply terms for GPAI Models.** Which models exist. The price per unit of use, which recurs for the life of the product and lands in cost of goods. Latency and availability. What the license permits. How long a given version stays supported. Every one of these is set by a supplier and revised on the supplier's schedule.
3. **Behavior of a specific GPAI Model version.** The same input can produce a different output after a supplier update. Two releases carrying the same product name are not the same object. This one reaches past the budget and into validation, which is what makes it different from the two above. A frozen benchmark result does not survive a supplier update unless you pinned the version and the supplier honors the pin.

People ordinarily call four different things "the model." What general-purpose systems can do at all. A supplier's model, which you buy. The trained component you ship. And the device software function that FDA authorizes. They are not the same object, and which one you mean decides what you control and what a change to it costs. Appendix A defines how I use each in the article.

The rate of change is measurable. A registry analysis of FDA authorizations of AI-enabled medical devices counted 1,430 cumulative authorizations through December 2025, of which 331 came in 2025 alone. The mean annual rate rose from under two per year through 2014 to roughly 264 per year across 2023 to 2025.

Put that against your own timeline. A program starting now and reaching market in eight years will launch into a field where most of the AI-enabled devices it competes with, is compared against, or is judged alongside were authorized after the program began. Your predicate landscape, your competitive set and the evidence reviewers have seen will all have changed while you built. None of that is a forecast. It is the current rate continued for the length of a program you have already committed to.

A milestone worth marking: *Through December 2024, a review of authorized AI-enabled devices found none built on a large language model. In December 2025 FDA cleared UpDoc V1.0, a prescription software device for insulin management, which its developer describes as the first Software as a Medical Device using patient-facing large language models. By mid-2026 the agency's breakthrough pipeline was filling with generative AI tools, and FDA had said it would begin tagging foundation-model-based devices in its published list. Eighteen months, from none to a cleared example and a stated intention to track the category..*

The envelope moving does not change what your product measures. Capability is what systems can do in general. Your performance is a measurement of one component, against one population, on defined inputs, in a defined workflow, against a defined endpoint. A general system getting better at a task in the abstract does not make your component better at your task, and it does not make your evidence weaker. A team that reads every model release as an existential event is treating a change in general capability as a change in its own numbers. The question below is about general capability. Your clinical evidence is about your numbers.

Competitive barriers have always mattered. What has changed is that one class of them is dissolving. When a task was hard, the difficulty was itself a barrier, and a team that could do what few others could held a position on that basis alone. That has been a real position for a long time. What the expanding capability envelope takes away is one kind of difficulty: the kind that consists

of a task being hard to learn to do. It does not touch difficulty that is physical, organizational, or economic. Manufacturing to tolerance at volume is still hard. Running a clean multi-site trial is still hard. Getting a hospital to change how it works is still hard. If your barrier is one of those, it is doing something the envelope may not be able to reach. The question below is therefore not a new question. What is new is how many of the answers that used to work no longer do.

Name the barrier, not the capability

Apply this test to your product. Ask:

What stops a competent competitor, using freely available general-purpose systems, from offering your product's core benefit within three years?

The question is deliberately about a barrier rather than about a technology. A barrier is a fact someone can check, rank, and argue with. "The capability will not get there" is none of those things, and it is the answer that ages worst.

These count as barriers. A sensor or mechanism you built. Physical access to the patient. A cleared indication. An integration that requires another party's cooperation. Contracted access to data that cannot be reproduced from public sources. Accumulated clinical evidence that cost years to generate. The willingness to carry liability for the result. Test each one the same way: a competitor with capital should not be able to reproduce it inside your window, and should not be able to know in advance how long it would take.

Data is the entry on that list most often claimed and least often checked. Large volumes of common labeled data are becoming commodity. Longitudinal, multimodal, outcome-linked data drawn from a population you have contractual access to is not. The two are easy to confuse from the inside, because both feel proprietary when you are the one holding them. Establish which one you have, and read the contract.

If the honest answer is that nothing stops them, change the plan now, while the cost of changing is still low. Being somewhat better than free is not a defensible position across a seven year window.

If you are not sure about a barrier, ask a second question.

What would have to be true for this task not to commoditize, and are those things true?

That turns a comfortable answer into a set of conditions, and conditions can be checked. Those conditions belong in the tracking list.

Name your software supplier dependency correctly

This section applies if your device contains third-party software you did not write. A GPAI Model reached by API is the case this section is about, but the same obligation covers any bought component: a commercial library, an operating system, a vendor SDK. If everything in your device was written by your team, skip to the next section.

One distinction decides whether any of this applies. A model your product calls while running is inside your device, whatever machine it executes on. A model you used to build the component and no longer call is not. That second case is now the more common one, and it is not free of consequences: whether the license covers what you derived, and whether you could rebuild the

component without access to the model you built it from, are both questions worth answering once and recording.

Two frameworks already cover this: software of unknown provenance under IEC 62304, and off-the-shelf software under FDA's. Which applies, and whether the SOUP definition is met, depends on the model's provenance and on what the supplier discloses. That is a determination your quality system makes and records. The failure is never having asked, and finding out during a submission or an audit that nobody did.

The supplier versions it, deprecates it, reprices it, and changes its behavior on their schedule, not yours. Answer one question in writing: what happens to the device when the supplier retires the version you validated against? If you cannot answer it, you have a demonstration, not a design.

Placement follows. A GPAI Model dependency belongs in Key Partners and in the risk register. It does not belong in Competitive Advantage. Access to a model your competitors can obtain on the same terms is not, by itself, a position.

The awkward part is not that you depend on someone. It is that the dependency sits outside your control and inside your responsibility. FDA authorizes your device and your name is on it. When the supplier changes something, nobody at the agency, and nobody in a deposition, is going to treat that as their problem.

Make the trained component replaceable

This section applies if your device contains a trained component, whether you built it yourself or bought it. If nothing in your product was learned from data, skip to the next section.

Assume you will replace the trained component several times during the product's life. Your data will improve, performance will drift as sites and populations change, postmarket signals will show you something validation did not, and a supplier, if you have one, will move on its own schedule. Some of those you will choose and some will choose you. A component trained at one point against one population does not stay correct for ten years.

Two separate things determine whether you can act when one arrives.

The first is architectural. Decide early exactly what goes into the component and exactly what comes out, and write that boundary down as a fixed specification. Everything else in the product talks to the boundary rather than to the component behind it. If the component's particulars are instead wired into the clinical claim, the screen design and the data preparation, then swapping it is not a swap. It is a redesign, and it will not happen.

The second is regulatory, and it has two levels that do not coincide. FDA authorizes the device software function, with its intended use and indications. It regulates the whole product. A Predetermined Change Control Plan operates on the inner scope: modifications specified in an authorized plan can be made without a separate submission, and modifications outside it are assessed under the ordinary rules for device changes. FDA's current final guidance was published in August 2025.

Both locks have to be open, and neither is cheap to add afterward. One without the other leaves you ready on paper and stuck in practice, or the reverse. The PCCP is written into the submission

and the boundary into the design: different people, different phase, different document. Unless something in your process ties them together, nothing forces you to have both.

Skip both and you ship the component you cleared with and you keep it, while the capability envelope expands and your supplier moves on. That is the actual mechanism by which a cleared product becomes obsolete on the shelf, and it is why this belongs in year one of a long program rather than year six. Being able to replace the component is not the same as knowing when to, which is a monitoring question, and the tracking section covers it.

The slow layer: assumptions treated as facts

The slow layer is the part you can see coming. Regulatory pathways, payment, evidence standards, procurement, workflow, staffing, liability, the mechanics of a procedure and the hardware that performs it all move through institutions with published processes, known participants and long cycle times. Most of these changes do not happen overnight, and they leave observable signals.

That does not make the changes predictable. **Direction is knowable. Timing and degree are not.** Evidence bars rise; nobody can tell you by how much or before which review. Care migrates to lower-cost settings; nobody can tell you when your procedure follows. Knowing the direction and guessing the date is how a team ends up with a study designed for a bar that moved while it ran.

The method here is not forecasting and not planning around a date. The direction is not in doubt. The timing is. So for each assumption, state what your plan depends on, and name the indicators you would watch for. Then watch them. One indicator is noise. Several pointing the same way are a change underway, and that is the point at which you have both a reason to plan and enough evidence to plan against. The aim is not to predict the shift. It is to have a response ready before it forces one.

Four examples follow. They are not a checklist and they are not in priority order; which one binds hardest depends on your product. Three of them hide the same way: they were examined once, an answer was reached, and the answer stopped looking like an assumption. Closure is what hides them. The fourth, the workflow, was never written down in the first place. One needs reopening. The other needs stating.

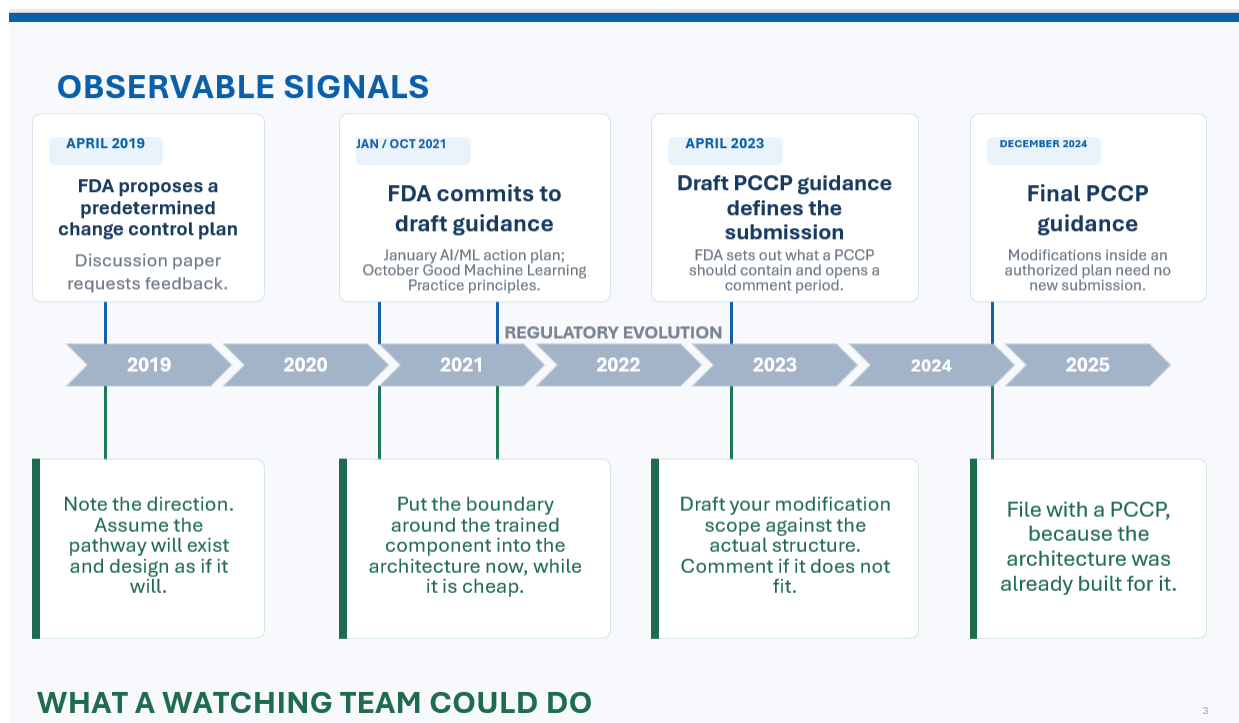
What you are assuming	Leading indicators	The question to revisit
The comparator. That years from now, when you have results, the thing you are measured against will still be the thing you are measured against today. This problem is well developed in oncology drug trials, where a control arm chosen correctly at the start can be outdated by the time accrual finishes. It	A competitor clearing in your indication. A guideline updating. A new therapy arriving upstream of your procedure. A code changing in a way that alters what the alternative costs.	Does your claim still support the price, the label and the adoption case against the comparison you would actually face? Answered early it changes the design. Answered late it changes only the interpretation.

What you are assuming	Leading indicators	The question to revisit
transfers to device and digital medicine programs, where the same thing happens for different reasons and is discussed far less.		
Payment. That the payment mechanism you confirmed is the one that will exist when you launch.	A code revalued. A coverage determination reversed or narrowed. A site-neutral rule. A payer consolidating. A budget moving from capital to operating. A competitor setting a reference price below the one your model assumes.	Who holds the budget, and what would move it? Which mechanism are you counting on, and what would make it unavailable? What price does your model assume, and what would make that price indefensible?
The workflow. That your product enters the workflow you designed it for.	Care migrating from hospital to ambulatory to home. Purchasing consolidating. Workforce shortages changing who performs a procedure. Adjacent therapies changing procedure volume. None of these is technological, which is why a team scanning for technology risk does not see them coming. If you sit in the information workflow, add ambient documentation, machine-generated differentials arriving with patients, and automated prior authorization.	Write one paragraph describing the workflow your product sits inside in the year you launch, and mark the sentences you are betting the company on. That paragraph is the assumption, and marking it is what makes it checkable.
The evidence bar. That the evidence which cleared a comparable product is the evidence that will clear yours.	Recent clearances in your category and what they were required to show. Guidance revisions. Advisory committee discussions. What payers have started demanding beyond what regulators require, which usually moves first and pulls the rest along.	Is the study designed against the bar you will face, or the one that cleared a product five years ago? Bars move one way: sufficient becomes suggestive, single-site becomes multi-site, retrospective becomes prospective, surrogate endpoints get questioned.

Two of these hide better than the others. Your trial result is not a property of your product; it is the difference between your product and whatever you compared it to, so the comparator disappears into a number that looks like a fact about you. And predicate reasoning makes the evidence bar feel like a property of the pathway rather than a standard reviewers move.

A worked example, because observable sounds abstract until you see one. The Predetermined Change Control Plan is now the mechanism that lets a team update a trained component without a new submission. It did not appear in December 2024. FDA proposed the idea in a discussion paper in April 2019, committed to guidance in an action plan in January 2021, and published a draft in April 2023. Five years, four documents, each one narrowing, all of them public and most open for comment. A team watching had time to design the architecture the pathway would require. A team not watching met it at submission, when the architecture was already fixed.

Figure 2: Five years of watching and acting as PCCPs evolve



Tracking the assumptions

For your company, choose the few assumptions that are actually load-bearing. Naming them is the easy part. A list written once and never reopened is worse than none, because it retires the worry without removing the exposure. Three things make it stick, and none of them is about the list.

1. **Give each observable an owner by name.** Not a function, not the team. One person who would be asked why nobody noticed. Assumptions owned by everyone are checked by no one, and a rotating owner is the same thing slower.

2. **Put the review on an existing calendar.** Quarterly, attached to a meeting that already happens for another reason. A standalone assumption review is the first meeting cancelled in a busy quarter, and busy quarters are when assumptions move.
3. **Write the response before the observable fires.** This is the one that has to be done before you need it, and the only one you cannot add later. When a competitor clears or a code is revalued, the discussion happens under pressure, with a founder defending a plan they authored. Deciding in advance what a given signal means, and what you will do about it, converts a fight into a lookup. It also surfaces a useful fact early: if you cannot say what you would do, you do not have a plan that depends on that assumption, you have a hope.

One thing has changed about the cost of this. Watching competitor clearances, guidance revisions, advisory committee discussions, coverage determinations and code changes is broad, continuous, low-value-per-item work, which is why it gets dropped. It is also the kind of work today's tools do well, and it belongs in the Immediate layer above. That does not change who owns the conclusion: a signal still has to be read by someone who knows what it would mean for your product. It changes whether anyone is looking at all, which is the first failure mode below.

Sort widely, then track the few you cannot afford to be wrong about. A few assumptions, not twenty risks. Load-bearing means the company has no product if one is false. A list long enough to be complete is long enough to be ignored.

Two failure modes worth watching for. I have not run this long enough to have seen the first, but I would expect a check that returns "no change" every quarter for two years to mean nobody is looking, or that the observable is wrong, rather than that the world is still. Those need different fixes: the first is the owner or the cadence, the second is the list. The second failure mode needs no field evidence. An assumption for which nobody can find an observable may or may not be stable, which is the problem, and it should be rewritten until something about it can be looked at.

Investors will form a view whether or not they ask

Anyone running diligence on your company will reach a conclusion about your AI exposure. Many will never ask you about it directly. Some do not have the vocabulary for the medical device specifics. Some ask a general question, receive a general answer, and move on. Some raise it after the meeting, in a partner discussion you are not in.

Silence in the room is not absence of the question. It means the question is being answered without you.

Bring it yourself, on one page. Which layer your exposure actually sits in. What stops a competitor today. What you would do if that stopped working. What you have already changed in how the team works, with one example. That page converts a lurking objection into evidence that you have already thought about the thing they were going to wonder about anyway.

Do not write an AI section that is a forecast about where the technology goes. That is the error this article opened with, an investor who knows the sector will read it as hand waving, and it is worse than saying nothing.

Feedback?

What was not clear, or what did you have to read twice? What is missing that you expected to find? Where does the framework not match your experience? And if you think the sorting itself is wrong, three layers, or these layers, that is the harder answer to give and the one I am asking for. Those of you who know me know I give and respect blunt feedback. Anonymous responses are fine, as noted at the top.

Contact: jcollins11@mgh.harvard.edu

Appendix A: Terms

These terms are used in a specific way in this article. Nearly all of them are things people ordinarily call "the model", which is the reason for defining them at all. That one word gets used for what any system can do, for the supplier's product, for the piece you ship, for the regulated device, and for a number on a slide. Keeping the five apart is what lets you see which of them your business actually depends on. The bare word *model* is left to its business sense, as in business model.

Capability envelope. The set of tasks general-purpose AI systems can perform at a given point in time. Not a product. Not owned by anyone. Expands on the fast clock, discontinuously. This is the referent when assessing whether commodity capability will overtake a product's differentiation. Distinct from the capability of your own device, which is a property of a specific trained component and is measured, not estimated.

GPAI Model. A specific third-party general-purpose AI model, reached by API or licensed for local deployment. *General-purpose AI model* is a defined term in the EU AI Act, which distinguishes the GPAI model from the GPAI system built on top of it. Preferred over *foundation model*, which names a training regime and embeds a claim about being a load-bearing base. Preferred over *frontier model*, which names a moving policy subset rather than a class.

SOUP (software of unknown provenance). IEC 62304 term for software incorporated into a medical device that was not developed under a compliant process and whose internals cannot be fully inspected or verified. FDA addresses the same category as off-the-shelf software in a medical device. A GPAI Model used inside a device will usually need to be managed under this framework or FDA's off-the-shelf software framework, with the additional properties of a network dependency and a supplier who can change behavior without notice. Which framework applies, and whether the SOUP definition is met, depends on the model's provenance and on what the supplier discloses. This is the role noun. It is the one that changes what happens in a design review.

Trained component. The specific trained part of the software that ships inside your device. It covers what the component learned during development, how it was built, how incoming data is prepared before it is used, and where the cut-offs sit that turn a score into a result. This is what gets versioned, frozen, and validated. If the component is built on top of a GPAI Model rather than trained from your own data, it incorporates SOUP and inherits the supplier risk. Teams routinely call such a system *our model*. It is not theirs. A useful check is to ask which parts of the component you can reproduce from inputs you control.

Device software function. The object your marketing submission authorizes. FDA does not clear a trained component. It clears a device with a stated intended use and indications for use, of which the trained component is a component. Conflating the trained component with the device software function is the expensive error, because it hides the only question that matters when you propose an update: does the change stay inside the cleared intended use and inside the authorized PCCP?

Trained component performance. A measurement of a specified trained component against a specified dataset and endpoint under specified conditions. It is a property of the pairing, not of the component alone. *Our model is 94 percent accurate* is not a statement. *This component achieved 94 percent sensitivity on a test set it never saw during development, under these reader conditions* is. Naming the object measured also exposes a common substitution: a number lifted from a supplier benchmark is GPAI Model performance, which is a different measurement of a different object.

PCCP (Predetermined Change Control Plan). A prespecified description of intended modifications to an AI-enabled device software function, the methodology for developing and validating them, and an assessment of their impact, reviewed as part of the marketing submission. Modifications implemented within an authorized PCCP do not require a new marketing submission.

Appendix B: Where this lands on the canvas

For teams using the Healthcare Business Model Canvas, at <https://www.gaits.org/hbmc>. Skip this if you do not use one. Every section of this article tests a claim you have already written down somewhere on the canvas. Nothing here adds a block. It gives you a reason to reopen ones you have already filled in.

Section	**Canvas block**	**What it tests**
Name the barrier	3a Competitive Advantage 7a Channels 7b Customer Relationships	Whether a funded competitor could reproduce the advantage inside your window
Name your supplier dependency	6c Key Partners	Whether a GPAI Model dependency is recorded as a partner rather than an advantage
Make the trained component replaceable	6a Key Resources	Whether you can change the component without a redesign or a new submission
The comparator	3a Competitive Advantage 3b Value Quantification 4b Market Access 4c Advisory Bodies	Whether the claim survives the comparison you will face when you report
Payment	5b Transaction Model 8a Revenue Stream	Whether a payer or budget holder, a mechanism and a budget line are named
The workflow	1 Unmet Need 2a Users	Whether the workflow and need assumed is the one that will exist at launch
The evidence bar	3a Competitive Advantage 3b Value Quantification	Whether the study is designed against the bar you will face rather than today's
Use the tools now	8b Cost Structure	Whether the plan reflects what the team can now do per person

Appendix C: Where this comes from

Almost nothing in this framework is new on its own. What may be useful is the assembly, and which parts of it apply to a device program.

Layers that change at different rates. Stewart Brand, *The Clock of the Long Now* (1999), building on Frank Duffy's shearing layers in architecture. Brand's point is that trouble comes from binding fast and slow layers together. Sean Savage applied the same idea to AI product design in "AI Pace Layers" (2025), layering the components of a product rather than the assumptions behind a business: <https://uxdesign.cc/ai-pace-layers-a-framework-for-resilient-product-design-530c19e3f9b0>

Matching strategy to the rate of change. Charles Fine, *Clockspeed: Winning Industry Control in the Age of Temporary Advantage* (1998). Simon Wardley's mapping work tracks components moving toward commodity at different speeds: <https://learnwardleymapping.com>

Naming assumptions and watching for them to fail. The slow layer's method is closest to Assumption-Based Planning, developed by James Dewar and colleagues at RAND. See Dewar, Builder, Hix and Levin, *Assumption-Based Planning: A Planning Tool for Very Uncertain Times*, RAND MR-114 (1993), available at https://www.rand.org/pubs/monograph_reports/MR114.html, and Dewar's *Assumption-Based Planning: A Tool for Reducing Avoidable Surprises*, Cambridge University Press (2002). ABP asks which assumptions a plan is load-bearing on, sets what Dewar calls signposts, meaning observable events that would show an assumption failing, and pairs them with actions prepared in advance. I use "observable" rather than "signpost," but the debt is his.

Barriers a competitor cannot reliably reproduce. The second half of the barrier test, that a competitor should not be able to know in advance how long reproduction would take, is Lippman and Rumelt's uncertain imitability, *Bell Journal of Economics* 13(2), 418 to 438 (1982), and the causal ambiguity literature that followed it. The first half, that model access is not a moat, is close to consensus in current writing on AI defensibility.

Model versus system. The distinction between a general-purpose AI model and the system built on it is drawn in law by the EU AI Act, which separates the two and assigns different obligations to each. See Article 3 and Recital 97: <https://eur-lex.europa.eu/eli/reg/2024/1689/oj> Appendix A extends that distinction rather than inventing it.

The comparator problem. Well developed in oncology drug trial methodology, including work on control arms that were appropriate at the start of a trial and outdated by the end. See the analysis of control-arm adequacy in oncology randomised trials, *European Journal of Cancer* (2023): <https://www.sciencedirect.com/science/article/abs/pii/S0959804923002502>

Predetermined Change Control Plans. FDA, *Marketing Submission Recommendations for a Predetermined Change Control Plan for Artificial Intelligence-Enabled Device Software Functions*, final guidance, December 2024: <https://www.fda.gov/regulatory-information/search-fda-guidance-documents/marketing-submission-recommendations-predetermined-change-control-plan-artificial-intelligence>

Authorization counts. The figures in the fast layer are from a registry analysis of three decades of FDA authorizations of AI and ML-enabled medical devices, covering 1995 to December 2025: <https://pmc.ncbi.nlm.nih.gov/articles/PMC13464356/>

What I have added is the sorting, the four assumptions I think matter most in a device program, and the claim that most of this applies whether or not your product contains any AI.

The first cleared device using large language models. FDA 510(k) K253281, UpDoc V1.0, cleared 23 December 2025. The clearance record is searchable in FDA's 510(k) database at <https://www.accessdata.fda.gov/scripts/cdrh/cfdocs/cfpmn/pmn.cfm>